DESENVOLVIMENTO MOBILE · MÓDULO 07 · TRILHA KOTLIN · NÍVEL INICIANTE

Estado, Navegação e Animações

Estado em Compose, recomposição, state hoisting, efeitos colaterais, ciclo de vida, Navigation 3 e animações

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Criar estado com `remember` e `mutableStateOf` e entender a recomposição.
- Preservar estado em rotações com `rememberSaveable`.
- Aplicar *state hoisting* para criar componentes reutilizáveis e testáveis.
- Usar efeitos colaterais com segurança (`LaunchedEffect`, `DisposableEffect`...).
- Explicar o ciclo de vida da Activity e coletar fluxos respeitando-o.
- Implementar navegação entre telas com **Navigation 3**.
- Animar mudanças de estado com as APIs de animação do Compose.

Roteiro

Neste módulo

- Fundamentos: estado, ciclo de vida e navegação
- Estado e recomposição
- State hoisting
- Efeitos colaterais
- Ciclo de vida
- Navegação com Navigation 3
- Animações
- Fechamento

Fundamentos: estado, ciclo de vida e navegação

Estado: o que a tela precisa lembrar

Estado

Todo valor que pode mudar com o tempo e influencia o que a tela mostra. A interface é uma função dele: **UI = f(estado)**.

Tipo	Exemplo no LabMonitor	Onde mora
Efêmero	painel expandido, rolagem, aba ativa	no componente
Da tela	bancadas carregadas, filtro, erro	no dono da tela
Do app	usuário logado, tema escolhido	em um serviço compartilhado
Persistente	reservas, preferências	banco ou disco (Módulo 10)

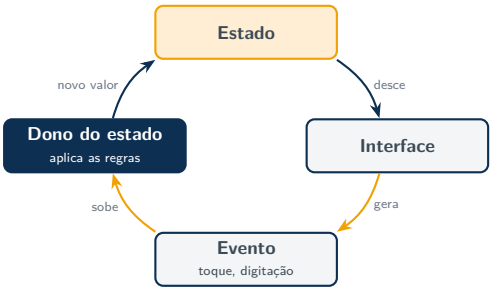
Estado derivado

Valor calculado a partir de outros, como o total de bancadas livres. Não se guarda: calcula-se, para nunca ficar desatualizado.

Fonte única da verdade

Cada dado tem um único dono que pode alterá-lo; os demais apenas leem. Duas cópias editáveis do mesmo dado acabam divergindo.

Fluxo unidirecional de dados



Fluxo unidirecional (UDF)

O estado desce do dono até a interface; os eventos sobem da interface até o dono. **Só o dono altera o estado**.

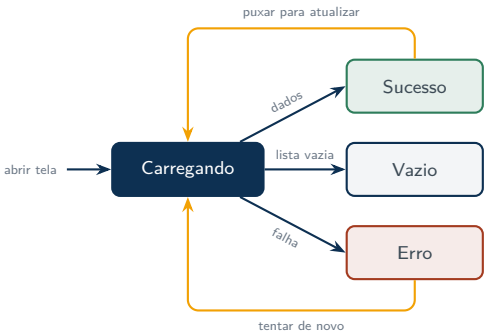
Vantagens

Previsível: cada mudança tem um único caminho. Fácil de depurar: registre eventos e estados. Fácil de testar: um evento entra, um estado sai.

Um pouco de história

A Arquitetura Elm, o Flux do Facebook (2014) e o Redux (2015) popularizaram o modelo, hoje padrão no Compose, no Flutter e no SwiftUI.

A tela como máquina de estados



Máquina de estados finitos

Conjunto fechado de estados e de transições permitidas, disparadas por eventos. A tela está sempre em **exatamente um** estado.

Estados impossíveis

Três booleanos (`carregando`, `erro`, `vazio`) geram 8 combinações, a maioria sem sentido. Uma classe selada com quatro casos admite só os quatro válidos.

Dica

Kotlin e Dart 3 têm classes seladas: o compilador obriga o `when` ou o `switch` a tratar todos os casos.

O que sobrevive a quê

Onde o valor está	nova recomposição	rotação da tela	morte do processo	app fechado pelo usuário
variável local dentro da função de interface	não	não	não	não
`remember` (Compose)	**sim**	não	não	não
`rememberSaveable` (Compose)	**sim**	**sim**	**sim**	não
`ViewModel` (Android)	**sim**	**sim**	não[1]	não
`State` de um `StatefulWidget` (Flutter)	**sim**	**sim**[2]	não	não
`RestorationMixin` (Flutter)	**sim**	**sim**	**sim**	não
banco de dados, arquivo, preferências (Módulo 10)	**sim**	**sim**	**sim**	**sim**

[1] sim, com `SavedStateHandle`. [2] o Flutter não recria a tela ao girar.

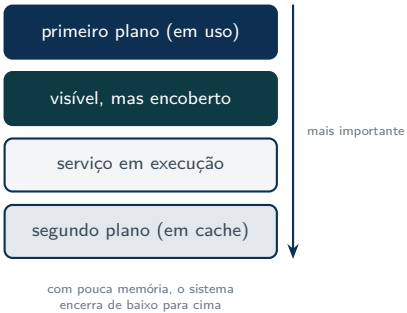
Morte do processo

Em segundo plano, o sistema pode encerrar o app para liberar memória. Ao voltar, o usuário espera a tela como a deixou: o sistema a recria e devolve o pequeno estado salvo.

Como testar

Ative "Não manter atividades" nas opções do desenvolvedor do Android, ou rode `adb shell am kill <pacote>` com o app em segundo plano.

Ciclo de vida e prioridade dos processos



Ciclo de vida

Sequência de estados por que passam o app e cada tela, controlada pelo sistema: criado, visível, em primeiro plano, em segundo plano, destruído. Cada transição é avisada por *callbacks*.

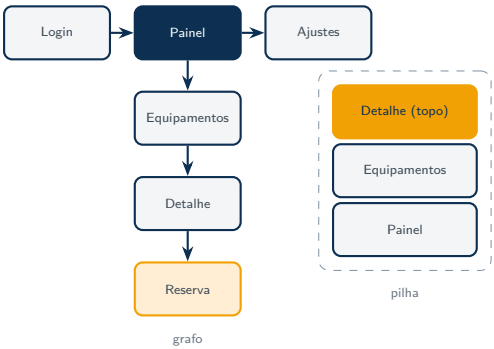
O que fazer nas transições

Ao sair do primeiro plano: pare câmera, sensores e animações e salve rascunhos. Ao voltar: retome e atualize os dados.

Por que o sistema encerra apps

A memória do celular é limitada, e o sistema prefere encerrar apps em segundo plano a deixar o aparelho lento: no Android, o *low memory killer*; no iOS, o *jetsam*.

Navegação: grafo de destinos e pilha de retorno



Grafo e pilha

Destinos (telas) são **nós**; as ligações permitidas, **arestas**. A pilha de retorno é LIFO: navegar empilha, voltar desempilha.

Princípios

Destino inicial fixo; o voltar nunca surpreende; cada aba da barra inferior tem a própria pilha.

Voltar preditivo

Com `targetSdk` 36 no Android 16, o gesto mostra a tela anterior antes de o dedo soltar e `onBackPressed` não é mais chamado: use `BackHandler` ou `PopScope`.

Navegação declarativa e links profundos

Imperativa *versus* declarativa

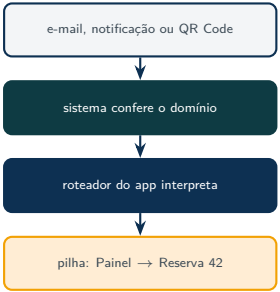
Imperativa: comandos `push` e `pop`. Declarativa: a pilha é um **estado** (uma lista de destinos) e o que aparece é função dela, como na interface (Navigation 3, go_router).

Link profundo (*deep link*)

Endereço que abre o app direto em um destino, como `https://labmonitor.app/reserva/42`. O app monta uma pilha sintética (Painel, Reserva) para o voltar fazer sentido.

Atenção

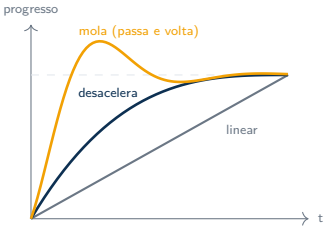
Parâmetros de um link vêm de fora: valide-os (a reserva existe? este usuário pode vê-la?).



Links verificados

App Links (Android, arquivo `assetlinks.json`) e *Universal Links* (iOS, `apple-app-site-association`) provam que o site e o app têm o mesmo dono.

Movimento: interpolação, curvas e molas



Interpolação

A cada quadro: `valor = início + (fim − início) × curva(t)`, com `t` indo de 0 a 1 ao longo da animação.

Curvas e molas

Curvas de aceleração (*easing*) imitam objetos reais: desaceleram ao chegar. **Molas** são definidas por rigidez e amortecimento, não por duração, e podem ser interrompidas sem saltos. O Material 3 Expressive (2025) adotou molas como base.

Movimento com propósito

Anime para orientar: de onde veio a tela, o que mudou. Transições de 150 a 400 ms mantêm o app ágil.

Acessibilidade

Respeite a opção do sistema de remover ou reduzir animações: movimento excessivo causa desconforto em algumas pessoas.

Estado e recomposição

O que é estado

Estado é qualquer valor que pode mudar com o tempo e afeta o que aparece na tela: o texto digitado, a lista carregada, se um diálogo está aberto, a última leitura do sensor.

Não funciona!

```
@Composable
fun Contador() {
  var cliques = 0          // variável comum
  Button(onClick = { cliques++ }) {
    Text("Cliques: $cliques")  // nunca muda
  }
}
```

Erro comum

Mudar uma variável comum não avisa o Compose, e a cada recomposição ela volta a 0.

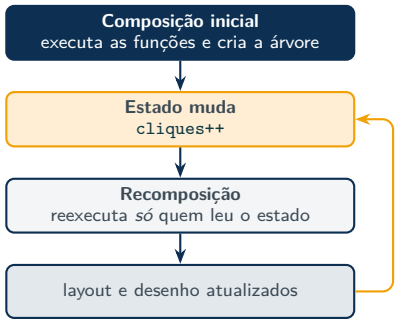
Correto

```
@Composable
fun Contador() {
  var cliques by remember { mutableStateOf(0) }
  Button(onClick = { cliques++ }) {
    Text("Cliques: $cliques")
  }
}
```

Duas peças

- `mutableStateOf`: valor **observável**; ler registra dependência, escrever agenda recomposição.
- `remember`: guarda o objeto entre recomposições.

Recomposição



Propriedades

- É **inteligente**: funções cujos parâmetros não mudaram podem ser puladas.
- Pode acontecer **a cada quadro** (ex.: animações).
- Pode ser **cancelada** e refeita.

Dica

O *Layout Inspector* mostra quantas vezes cada composable recompôs. Números muito altos indicam problema de desempenho (Módulo 14).

`remember` *versus* `rememberSaveable`

Sobrevivendo à rotação

```
// perdido ao girar a tela
var filtro by remember { mutableStateOf("") }

// salvo no Bundle: sobrevive à rotação e à
// morte do processo em segundo plano
var busca by rememberSaveable { mutableStateOf("") }

// estado derivado: só recalcula quando muda
val visiveis by remember(itens) {
  derivedStateOf { itens.filter { busca in it.descricao } }
}

// listas observáveis
val selecionados = remember { mutableStateListOf<String>()
      }
```

	recomposição	rotação
variável comum	perde	perde
`remember`	mantém	perde
`rememberSaveable`	mantém	mantém
ViewModel	mantém	mantém

Atenção

`rememberSaveable` só aceita tipos que cabem num `Bundle` (primitivos, `String`, `Parcelable`). Para objetos maiores, use um ViewModel (Módulo 08).

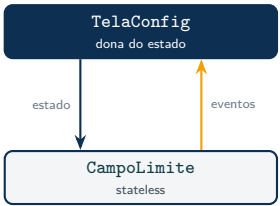
State hoisting

Elevando o estado (*state hoisting*)

Componente sem estado (stateless)

```
@Composable
fun CampoLimite(
  valor: String,                    // estado desce
  onValorChange: (String) -> Unit, // evento sobe
  modifier: Modifier = Modifier
) {
  OutlinedTextField(
    value = valor,
    onValueChange = onValorChange,
    label = { Text("Limite (°C)") },
    modifier = modifier
  )
}

// Quem usa decide onde o estado mora
@Composable
fun TelaConfig() {
  var limite by rememberSaveable { mutableStateOf("30") }
  CampoLimite(limite, onValorChange = { limite = it })
}
```



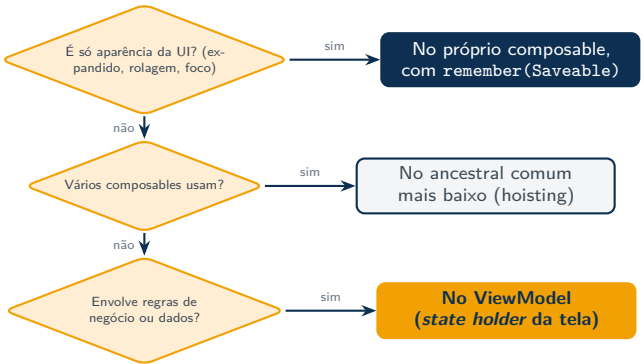
Fluxo unidirecional de dados (UDF)

O estado desce como parâmetro e os eventos sobem como lambdas. Há uma única **fonte da verdade**.

Vantagens

Componente reutilizável, fácil de testar e de visualizar em `@Preview`.

Onde o estado deve morar?



Dica

Regra prática: eleve o estado só até onde for necessário, nem mais nem menos.

Efeitos colaterais

APIs de efeitos

LaunchedEffect e DisposableEffect

```
@Composable
fun Monitor(bancadaId: Int, vm: PainelViewModel) {
  // roda quando entra na tela e quando a chave muda
  LaunchedEffect(bancadaId) {
    vm.carregar(bancadaId)   // coroutine
  }

  val lifecycle = LocalLifecycleOwner.current.lifecycle
  DisposableEffect(lifecycle) {
    val obs = LifecycleEventObserver { _, e ->
      if (e == Lifecycle.Event.ON_RESUME) vm.reconectar()
    }
    lifecycle.addObserver(obs)
    onDispose { lifecycle.removeObserver(obs) }
  }
}
```

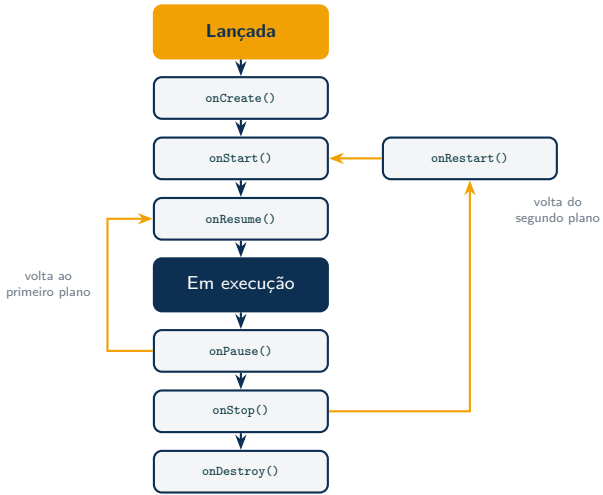
API	Quando usar
`LaunchedEffect(chave)`	rodar `suspend` ao entrar ou quando a chave muda
`rememberCoroutineScope`	iniciar coroutine a partir de um clique
`DisposableEffect`	registrar e remover *listeners*
`SideEffect`	sincronizar com código não-Compose a cada recomposição
`rememberUpdatedState`	usar o valor mais recente dentro de um efeito longo
`produceState`	converter fonte externa em estado
`derivedStateOf`	estado calculado que muda menos que a origem

Atenção

`LaunchedEffect(Unit)` roda uma única vez enquanto o composable estiver na tela.

Ciclo de vida

Ciclo de vida da Activity



Estado	Significado
Created	criada, ainda invisível
Started	visível
Resumed	em primeiro plano, recebe toques
Paused	parcialmente encoberta
Stopped	invisível (segundo plano)

Mudanças de configuração

Girar a tela, trocar idioma ou tema **destrói e recria** a Activity. Por isso o estado precisa estar em `rememberSaveable` ou no ViewModel.

Morte do processo

Em segundo plano, o sistema pode encerrar o app para liberar memória. Ao voltar, só o estado salvo sobrevive.

Coletando fluxos com segurança

TelaPainel.kt

```
@Composable
fun TelaPainel(vm: PainelViewModel = viewModel()) {
  // para de coletar quando o app vai para o
  // segundo plano (abaixo de STARTED)
  val estado by vm.estado.collectAsStateWithLifecycle()

  when (val e = estado) {
    PainelUiState.Carregando -> CircularProgressIndicator()
    is PainelUiState.Erro -> Text(e.mensagem)
    is PainelUiState.Pronto -> Painel(
      leituras = e.leituras,
      onAtualizar = vm::atualizar
    )
  }
}
```

`collectAsStateWithLifecycle`

Converte `StateFlow` em `State` do Compose e **pausa a coleta** quando a tela não está visível, economizando bateria, rede e sensores.

Dependência

Vem do artefato `androidx.lifecycle:lifecycle-runtime-compose`.

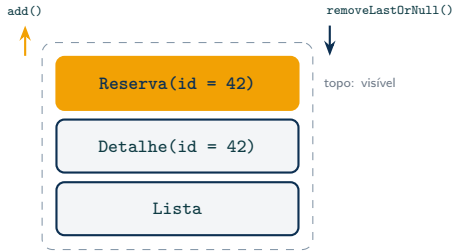
Erro comum

`collectAsState()` continua coletando em segundo plano. No Android, prefira a versão *WithLifecycle*.

Navegação com Navigation 3

Conceitos de navegação

- Apps Compose costumam ter **uma Activity** e várias telas (*destinos*).
- A **pilha de navegação** (*back stack*) guarda o histórico; o botão/gesto voltar remove o topo.
- **Navigation 3** (Jetpack, estável desde nov./2025) representa a pilha como uma **lista observável de chaves** controlada pelo próprio app.



Peças

`NavKey` (chave de cada tela) · `rememberNavBackStack` (a pilha) · `NavDisplay` (exibe o topo) · `entryProvider` (chave → tela).

Dica

Muitos projetos existentes usam **Navigation Compose** (Nav2, com `NavHost` e `NavController`). Os conceitos são os mesmos; ao final mostramos a equivalência.

Navigation 3: chaves e pilha

app/build.gradle.kts

```
plugins {
  alias(libs.plugins.kotlin.serialization)
}
dependencies {
  implementation(libs.androidx.navigation3.runtime)
  implementation(libs.androidx.navigation3.ui)
  implementation(
    libs.androidx.lifecycle.viewmodel.navigation3)
  implementation(libs.kotlinx.serialization.core)
}
```

Rotas.kt

```
@Serializable data object Lista : NavKey
@Serializable data class Detalhe(val id: Int) : NavKey
@Serializable data class Reserva(val id: Int) : NavKey
```

Chaves tipadas

Cada tela é uma classe `@Serializable` que implementa `NavKey`. Os argumentos são propriedades: sem strings de rota e sem erros de digitação.

Pilha persistente

`rememberNavBackStack(Lista)` cria a pilha com a tela inicial e a salva automaticamente em rotações e na morte do processo, por isso as chaves são serializáveis.

Dica

Passe só identificadores (`id`) nas chaves; a tela de destino busca os dados.

Navigation 3: NavDisplay

NavegacaoApp.kt

```
@Composable
fun NavegacaoApp() {
  val pilha = rememberNavBackStack(Lista)
  NavDisplay(
    backStack = pilha,
    onBack = { pilha.removeLastOrNull() },
    entryDecorators = listOf(
      rememberSaveableStateHolderNavEntryDecorator(),
      rememberViewModelStoreNavEntryDecorator()), // VM por tela
    entryProvider = entryProvider {
      entry<Lista> {
        TelaLista(onAbrir = { id -> pilha.add(Detalhe(id)) })
      }
      entry<Detalhe> { chave ->
        TelaDetalhe(chave.id,
          onReservar = { pilha.add(Reserva(chave.id)) })
      }
      entry<Reserva> { chave -> TelaReserva(chave.id) }
    }
  )
}
```

- **Navegar** = `pilha.add(chave)`.
- **Voltar** = `pilha.removeLastOrNull()`.
- O gesto de voltar do sistema chama `onBack`.
- Os decoradores salvam o estado de cada tela e criam um ViewModel por entrada da pilha.

Telas não conhecem a pilha

`TelaLista` recebe `onAbrir` como lambda, não a pilha. Assim ela continua reutilizável e testável.

Navegação principal com barra inferior

Abas principais

```
enum class Aba(val chave: NavKey, val rotulo: String,
               val icone: ImageVector) {
  PAINEL(Painel, "Painel", Icons.Default.Dashboard),
  EQUIP(Lista, "Equipamentos", Icons.Default.Memory),
  CONFIG(Config, "Ajustes", Icons.Default.Settings)
}

NavigationBar {
  Aba.entries.forEach { aba ->
    NavigationBarItem(
      selected = pilha.first() == aba.chave,
      onClick = {           // troca a raiz da pilha
        pilha.clear(); pilha.add(aba.chave)
      },
      icon = { Icon(aba.icone, null) },
      label = { Text(aba.rotulo) }
    )
  }
}
```

Uma pilha por aba

Em apps maiores, mantenha uma pilha para cada aba e exiba a da aba ativa. Assim cada aba lembra até onde o usuário navegou. Os *recipes* oficiais do Navigation 3 trazem esse padrão pronto.

Nav2 ↔ Nav3

`NavHost`	`NavDisplay`
`navigate(r)`	`pilha.add(r)`
`popBackStack()`	`removeLastOrNull()`
`composable<R>{}`	`entry<R>{}`
`toRoute<R>()`	a própria chave

Animações

Animações por valor alvo

CartaoMedidor.kt

```kotlin
@Composable
fun CartaoMedidor(temp: Double, expandido: Boolean) {
  val alerta = temp > 30
  val altura by animateDpAsState(
    if (expandido) 160.dp else 72.dp, label = "altura")
  val cor by animateColorAsState(
    if (alerta) Ambar else MaterialTheme.colorScheme.surface,
    label = "cor")
  Card(
    modifier = Modifier.height(altura),
    colors = CardDefaults.cardColors(containerColor = cor)
  ) {
    AnimatedContent(temp, label = "temp") { t ->
      Text("%.1f °C".format(t))
    }
    AnimatedVisibility(visible = alerta) {
      Text("Acima do limite!")
    }
  }
}
```

Valor alvo

Você informa o valor final; o Compose anima do valor atual até ele a cada mudança de estado. Cobre a maioria dos casos.

API	Anima
`animate*AsState`	um valor (`cor`, `Dp`, `Float`)
`AnimatedVisibility`	entrada e saída
`AnimatedContent`	troca de conteúdo
`animateContentSize`	mudança de tamanho

Dica

O parâmetro `label` identifica a animação no *Animation Preview* do Android Studio.

Animações contínuas, molas e transições

Ícone pulsando

```
@Composable
fun IconeAlerta() {
  val pulso = rememberInfiniteTransition(label = "pulso")
  val escala by pulso.animateFloat(
    initialValue = 1f, targetValue = 1.3f,
    animationSpec = infiniteRepeatable(
      tween(1000), RepeatMode.Reverse),
    label = "escala")
  Icon(Icons.Default.Warning, "Alerta", tint = Ambar,
    modifier = Modifier.graphicsLayer {
      scaleX = escala; scaleY = escala
    })
}
```

Painel com mola

```
val deslocamento by animateDpAsState(
  if (aberto) 0.dp else 300.dp,
  animationSpec = spring(
    dampingRatio = 0.5f,    // MediumBouncy
    stiffness = 200f),      // StiffnessLow
  label = "painel")
```

Mola *versus* `tween`

`tween` tem duração e curva fixas; `spring` segue a física e continua suave se o alvo mudar no meio do caminho.

Transições entre telas

`SharedTransitionLayout` faz a foto "voar" da lista ao detalhe; o Navigation 3 anima o voltar preditivo.

Fechamento

Resumo

Resumo

- `mutableStateOf` + `remember` criam estado observável; mudar o estado dispara a recomposição.
- `rememberSaveable` sobrevive à rotação; o ViewModel guarda o estado da tela.
- *State hoisting*: estado desce, eventos sobem, uma fonte da verdade.
- Efeitos colaterais só dentro de `LaunchedEffect`, `DisposableEffect` e afins.
- Rotação recria a Activity; colete fluxos com `collectAsStateWithLifecycle`.
- Navigation 3: chaves `NavKey`, pilha observável e `NavDisplay`.
- Animações: `animate*AsState`, `AnimatedVisibility`, `AnimatedContent` e molas.

Glossário (1/2)

Termo	Significado
Estado	valor que muda com o tempo e afeta o que a tela mostra
Estado derivado	valor calculado a partir de outros; não se guarda
Fonte única da verdade	cada dado tem um único dono que pode alterá-lo
Fluxo unidirecional	o estado desce até a interface; os eventos sobem até o dono
State hoisting	mover o estado para quem chama, deixando o componente sem estado
`remember`	guarda um valor entre recomposições
`rememberSaveable`	guarda também na rotação e na morte do processo
Efeito colateral	ação fora da descrição da tela: rede, registro, temporizador
`LaunchedEffect`	executa um efeito suspenso ligado à composição

Glossário (2/2)

Termo	Significado
Ciclo de vida	estados da `Activity`: *Created*, *Started*, *Resumed*...
Morte do processo	encerramento do app em segundo plano para liberar memória
Pilha de retorno	histórico LIFO dos destinos visitados
`NavKey`	chave que identifica um destino no Navigation 3
Link profundo	endereço que abre o app direto em um destino
Voltar preditivo	gesto que mostra a tela anterior antes de concluir o voltar
Mola (`spring`)	animação definida por rigidez e amortecimento, não por duração

Questões de revisão

1. Por que uma variável comum dentro de um composable não serve para guardar estado?
2. Explique o fluxo unidirecional de dados com o exemplo de um campo de busca.
3. Modele a tela de equipamentos como máquina de estados. Por que uma `sealed interface` e não três booleanos?
4. O que sobrevive à rotação e à morte do processo com `remember`, `rememberSaveable` e `ViewModel`?
5. O que deve acontecer com a leitura de sensores quando o app vai para o segundo plano? Por quê?
6. Desenhe a pilha de retorno depois de Painel, Equipamentos, Detalhe e um voltar.
7. Como um link profundo para a reserva 42 deve montar a pilha? Que validações fazer?
8. Por que uma animação com `spring` lida melhor com interrupções que um `tween`?

Exercícios

Exercício 1 — Rotação

Crie um formulário com três campos, um usando `remember` e dois com `rememberSaveable`. Gire o emulador e explique o que aconteceu.

Exercício 2 — Hoisting

Transforme um `SeletorDeFaixa` com estado interno (mínimo e máximo) em um componente *stateless* com `RangeSlider`. Escreva dois `@Preview`s com valores diferentes.

Exercício 3 — Ciclo de vida

Registre um `Log` em cada callback do ciclo de vida da `MainActivity`. Gire a tela, vá para a tela inicial e volte. Desenhe a sequência observada.

LabMonitor — Etapa 7

Implemente a navegação: **Painel**, **Equipamentos** (lista → detalhe → reserva) e **Ajustes** com `NavigationBar`. O formulário de reserva deve sobreviver à rotação.

Para aprofundar

- Estado e Jetpack Compose: `https://developer.android.com/develop/ui/compose/state`
- Onde elevar o estado: `https://developer.android.com/develop/ui/compose/state-hoisting`
- Efeitos colaterais: `https://developer.android.com/develop/ui/compose/side-effects`
- Ciclo de vida da Activity: `https://developer.android.com/guide/components/activities/activity-lifecycle`
- Navigation 3: `https://developer.android.com/guide/navigation/navigation-3`
- Recipes do Navigation 3: `https://github.com/android/nav3-recipes`

Respostas das questões de revisão

1. Ela volta ao valor inicial a cada recomposição, e alterá-la não avisa o Compose. Use `mutableStateOf` dentro de `remember`.
2. O `TextField` mostra `busca` (estado que desce) e chama `onBuscaChange` (evento que sobe); o `ViewModel` atualiza o estado, que volta à tela.
3. Estados `Carregando`, `Sucesso(itens)`, `Vazio` e `Erro(msg)`. A classe selada só admite esses quatro e o `when` obriga a tratar todos; booleanos permitem combinações impossíveis.
4. `remember`: nenhuma das duas. `rememberSaveable`: ambas. `ViewModel`: rotação sim; morte do processo só com `SavedStateHandle`.
5. Deve parar, porque gasta bateria e ninguém está vendo. `collectAsStateWithLifecycle` para de coletar quando a tela sai de *Started*.
6. Antes do voltar: Painel, Equipamentos, Detalhe (topo). Depois: Painel, Equipamentos (topo).
7. Com uma pilha sintética Painel, Reserva(42), para o voltar levar ao Painel. Validar se a reserva existe e se o usuário pode vê-la.
8. A mola parte da posição e da velocidade atuais e segue a física até o novo alvo; o `tween` recomeça com duração fixa e muda a velocidade de forma brusca.

Fim do Módulo 07

Próximo: Módulo 08 — Arquitetura de apps: MVVM, camadas, repositório, casos de uso e injeção de dependência com Hilt.